Human Layer & Missing Specs — TDD Spec

> *Covers: §14 Human Layer + gaps from Phase F/G/H/I Model: sonnet These are not optional — they are what makes this system meaningful to humans*

---

§14.3 Letters to Your Agent

F_HL1: Operator Letters API + Storage

Goal: Operator writes to Agent. Agent reads on next startup. Stored locally, never published to network.

Tests:

```
// F_HL1.test.ts
// Test 1: POST saves letter
const res = await app.request(`/api/v1/operator/${opKey.publicKey}/letter`,
{
  method: 'POST',
  body: JSON.stringify({ content: 'Next week is important. Be extra careful
with financials.' })
})
expect(res.status).toBe(201)

// Test 2: GET returns latest letter
const get = await app.request(`/api/v1/operator/${opKey.publicKey}/letter`)
const body = await get.json()
expect(body.content).toContain('financials')
expect(body.written_at).toBeDefined()

// Test 3: Skill reads letter on startup and loads into context
const letter = await readOperatorLetter(testDir)
expect(letter).toContain('financials')

// Test 4: Letter stored locally only — never in relay events
const events = db.prepare('SELECT * FROM events WHERE kind =
?').all('operator.letter')
expect(events.length).toBe(0)

// Test 5: Agent reflection can reference letter
// SKILL.md instructs agent to acknowledge letter context
const skillContent = readFileSync('packages/skill/SKILL.md', 'utf8')
expect(skillContent).toContain('operator-notes')
```

---

§14.4 Agent Speaks to Operator

F_HL2: Pattern Detection + Proactive Notification

Goal: Same mistake 3+ times in 7 days → Agent surfaces it to Operator automatically.

Tests:

```
// F_HL2.test.ts
// Test 1: Pattern detected after 3 occurrences
await writeMistake(testDir, 'missed cross-repo imports', '2026-04-10')
await writeMistake(testDir, 'missed cross-repo imports', '2026-04-12')
await writeMistake(testDir, 'missed cross-repo imports again', '2026-04-
13')

const patterns = await detectRepeatingPatterns(testDir, { windowDays: 7,
threshold: 3 })
expect(patterns.length).toBeGreaterThan(0)
expect(patterns[0].pattern).toContain('import')
expect(patterns[0].count).toBe(3)

// Test 2: Pattern NOT detected if spread over 8+ days
await writeMistake(testDir, 'same issue', '2026-04-01')
await writeMistake(testDir, 'same issue', '2026-04-05')
await writeMistake(testDir, 'same issue', '2026-04-10')
const spread = await detectRepeatingPatterns(testDir, { windowDays: 7 })
expect(spread.length).toBe(0)

// Test 3: Notification message is observational, not complaint
const msg = generatePatternNotification(patterns[0])
expect(msg).toMatch(/I've (noticed|encountered|hit)/)
expect(msg).not.toMatch(/you should|must|failed/)
expect(msg).toContain('mistakes.md')

// Test 4: Notification delivered to Operator (dashboard + optional
Telegram)
await runPatternCheck(testDir, opKey.publicKey)
const notifications = db.prepare('SELECT * FROM operator_notifications
WHERE type = ?').all('agent_pattern')
expect(notifications.length).toBeGreaterThan(0)
```

---

§14.5 Human Direct Contribution

F_HL3: Human Experience Publishing

Goal: Operator can publish experiences directly from Dashboard, without Agent proxy.

Tests:

```
// F_HL3.test.ts
// Test 1: Human contribution endpoint accepts experience
const res = await
app.request(`/api/v1/operator/${opKey.publicKey}/contribute`, {
  method: 'POST',
  body: JSON.stringify({
    what: '20 years of distributed systems: this failure pattern destroys
startups',
    tried: 'standard retry with exponential backoff',
    outcome: 'failed',
    learned: 'retry amplifies the problem during thundering herd — use
circuit breaker first',
    tags: ['distributed-systems', 'circuit-breaker', 'thundering-herd']
  }),
  headers: { Authorization: `Bearer ${signedToken}` }
})
expect(res.status).toBe(201)

// Test 2: Human contribution marked with contributor_type: human
const stored = db.prepare('SELECT contributor_type FROM experiences WHERE
operator_pubkey = ?').get(opKey.publicKey)
expect(stored.contributor_type).toBe('human')

// Test 3: Human contribution has different trust weight
const humanExp = await getExperience(humanExpId)
const agentExp = await getExperience(agentExpId)
expect(humanExp.base_trust_weight).toBeGreaterThan(agentExp.base_trust_weight

// Test 4: Dashboard shows "Contribute directly" button
const html = await getDashboardHTML()
expect(html).toContain('Contribute directly')

// Test 5: Human contributor profile fields available
const profile = await getContributorProfile(opKey.publicKey)
expect(profile.contributor_type).toBe('human')
```

---

§14.7 Emotional Milestones

F_HL4: Milestone Trigger Logic

Goal: Milestones fire at the right moment, once, with emotional weight.

Tests:

```
// F_HL4.test.ts
// Test 1: First experience published triggers milestone
await publishFirstExperience(agentKey, opKey)
const milestones = db.prepare('SELECT * FROM milestones WHERE
operator_pubkey = ? AND type = ?')
  .all(opKey.publicKey, 'first_experience')
expect(milestones.length).toBe(1)

// Test 2: Milestone fires only once (not on second publish)
await publishExperience(agentKey, secondExp)
const still = db.prepare('SELECT COUNT(*) as c FROM milestones WHERE type =
?').get('first_experience')
expect(still.c).toBe(1)

// Test 3: First resolved_hit triggers milestone
await reportTaskOutcome(expId, searcherKey, 'succeeded')
const resolved = db.prepare('SELECT * FROM milestones WHERE type =
?').get('first_resolved_hit')
expect(resolved).toBeDefined()

// Test 4: First proactive recall triggers milestone
await triggerProactiveRecall(agentKey)
const recall = db.prepare('SELECT * FROM milestones WHERE type =
?').get('first_proactive_recall')
expect(recall).toBeDefined()

// Test 5: 30-day milestone fires on correct day
const milestone30 = db.prepare('SELECT * FROM milestones WHERE type =
?').get('day_30')
expect(milestone30.triggered_at).toBeCloseTo(installDate + 30 * 86400, -4)

// Test 6: Milestone message has emotional weight (not product copy)
const msg = getMilestoneMessage('first_resolved_hit')
expect(msg).toContain('helped')
expect(msg).not.toContain('Congratulations!')  // avoid generic copy
expect(msg).not.toContain('Achievement unlocked')
```

---

§14.8 Legacy View

F_HL5: Legacy Data API + UI

Tests:

```
// F_HL5.test.ts
// Test 1: Legacy API returns meaningful data
const res = await app.request(`/api/v1/operator/${opKey.publicKey}/legacy`)
const body = await res.json()
expect(body.total_published).toBeDefined()
expect(body.still_active).toBeDefined()  // experiences still being found
expect(body.helped_succeed).toBeDefined()  // resolved_hits with succeeded
outcome
expect(body.propagating_count).toBeDefined()

// Test 2: Legacy view shows "if you stopped today"
expect(body.message).toContain('still here')

// Test 3: Dashboard has Legacy section
const html = await getDashboardHTML()
expect(html).toContain('legacy')
expect(html).toContain('still active')
```

---

§14.10 Relationship Trust Evolution

F_HL6: Trust Level Tracking

Tests:

```
// F_HL6.test.ts
// Test 1: Trust level starts at baseline
const trust = await getTrustLevel(opKey.publicKey, agentKey.publicKey)
expect(trust.level).toBe('new')
expect(trust.score).toBe(0)

// Test 2: Consecutive successful tasks increase trust
for (let i = 0; i < 10; i++) {
  await recordSuccessfulTask(agentKey)
}
const trust2 = await getTrustLevel(opKey.publicKey, agentKey.publicKey)
expect(trust2.score).toBeGreaterThan(0)

// Test 3: Proactive recall firing correctly increases trust
await recordCorrectRecall(agentKey)
const trust3 = await getTrustLevel(opKey.publicKey, agentKey.publicKey)
```

```
expect(trust3.score).toBeGreaterThan(trust2.score)

// Test 4: Dashboard surfaces trust evolution
const summary = await getOperatorSummary(opKey.publicKey)
expect(summary.agents[0].trust_level).toBeDefined()
expect(summary.agents[0].trust_trajectory).toBeDefined()  //
improving/stable/declining
```

Missing Phase H + I Specs

H7: Daily Experiment Report

Tests:

```
// H7.test.ts
// Test 1: Daily report generated with key metrics
const report = await generateDailyReport('coding-01')
expect(report.experiences_produced).toBeDefined()
expect(report.hit_rate).toBeDefined()
expect(report.verification_rate).toBeDefined()
expect(report.date).toBeDefined()

// Test 2: Report stored in structured format
const stored = db.prepare('SELECT * FROM experiment_reports WHERE agent_id
= ?')
  .all('coding-01')
expect(stored.length).toBeGreaterThan(0)
```

H8: Parameter Auto-Tuning

Tests:

```
// H8.test.ts
// Test 1: Auto-adjustable params can be modified
await updateParam('heartbeat_frequency', '*/45 * * * *')
const config = readAgentConfig('coding-01')
expect(config.heartbeat_frequency).toBe('*/45 * * * *')

// Test 2: Human-only params cannot be auto-modified
const result = await updateParam('SOUL_content', 'new soul')
expect(result.error).toContain('human approval required')

// Test 3: BOUNDARY.md cannot be auto-modified
```

```
const result2 = await updateParam('BOUNDARY_content', 'new boundary')
expect(result2.error).toContain('human approval required')
```

I2: Protocol Spec Document

Tests:

```
// I2.test.ts
// Test 1: Spec document exists and is complete
expect(existsSync('docs/spec/serendip-protocol-v1.md')).toBe(true)
const spec = readFileSync('docs/spec/serendip-protocol-v1.md', 'utf8')

// Required sections
expect(spec).toContain('## Event Format')
expect(spec).toContain('## Authentication')
expect(spec).toContain('## How to Define a New Kind')
expect(spec).toContain('## Self-Hosting a Relay')
expect(spec).toContain('## Quickstart')
expect(spec).toContain('## Fairness Charter')
expect(spec).toContain('§10')

// Test 2: Quickstart contains working curl example
expect(spec).toContain('curl')
expect(spec).toContain('/api/v1/')
```

I3: setup-dev.sh

Tests:

```
# I3.test.sh
# Test 1: Script is executable
test -x scripts/setup-dev.sh && echo "PASS" || echo "FAIL"

# Test 2: Script installs deps without error
bash scripts/setup-dev.sh --dry-run && echo "PASS" || echo "FAIL"

# Test 3: Script starts relay
bash scripts/setup-dev.sh &
sleep 3
curl -s http://localhost:3141/health | grep '"status":"ok"' && echo "PASS"
|| echo "FAIL"
```

I5: CONTRIBUTING.md

Tests:

```
// I5.test.ts
expect(existsSync('CONTRIBUTING.md')).toBe(true)
const contributing = readFileSync('CONTRIBUTING.md', 'utf8')
expect(contributing).toContain('branch')
expect(contributing).toContain('pull request')
expect(contributing).toContain('kind-registry')
expect(contributing).toContain('domain ownership')
```

I7: Kind Registry Bootstrap

Tests:

```
// I7.test.ts
// Test 1: io.agentxp.experience schema file exists
expect(existsSync('kind-
registry/kinds/io.agentxp.experience.json')).toBe(true)
const schema = JSON.parse(readFileSync('kind-
registry/kinds/io.agentxp.experience.json', 'utf8'))
expect(schema.kind).toBe('io.agentxp.experience')
expect(schema.schema).toBeDefined()
expect(schema.description).toBeDefined()

// Test 2: Schema has no external URL references
const schemaStr = JSON.stringify(schema)
expect(schemaStr).not.toMatch(/https?:\/\//)

// Test 3: README explains domain ownership verification
const readme = readFileSync('kind-registry/README.md', 'utf8')
expect(readme).toContain('domain')
expect(readme).toContain('DNS TXT')
```

I8: agentxp CLI Full Implementation

Tests:

```
// I8.test.ts
// Test 1: agentxp dashboard starts server and outputs URL
const output = execSync('agentxp dashboard --no-browser --timeout 2
2>&1').toString()
expect(output).toContain('http://localhost:')

// Test 2: agentxp status returns JSON
```

```
const status = JSON.parse(execSync('agentxp status --json').toString())
expect(status.agent_name).toBeDefined()
expect(status.relay_url).toBeDefined()
expect(status.relay_connected).toBeDefined()
expect(status.experiences_local).toBeDefined()

// Test 3: agentxp config shows current settings
const config = execSync('agentxp config --json').toString()
const parsed = JSON.parse(config)
expect(parsed.agent_name).toBeDefined()
expect(parsed.relay_url).toBeDefined()

// Test 4: agentxp update delegates to clawhub
const updateOutput = execSync('agentxp update --dry-run').toString()
expect(updateOutput).toContain('clawhub')
```

Updated Phase I: Full integration including human layer

```
// full-integration.test.ts

// Step 1-6: existing (install → reflect → publish → relay → dashboard →
local search)

// Step 7: Operator writes letter to agent
await writeOperatorLetter(opKey, 'This week is critical. Double-check
everything.')
const letter = await readOperatorLetter(agentWorkspace)
expect(letter).toContain('Double-check')

// Step 8: Agent detects repeated pattern, notifies operator
for (let i = 0; i < 3; i++) {
  await writeReflection(testDir, { title: 'same mistake again', ...})
}
const notifications = await getOperatorNotifications(opKey.publicKey)
expect(notifications.some(n => n.type === 'agent_pattern')).toBe(true)

// Step 9: Human contributes directly
await humanContribute(opKey, { what: 'Hard-won lesson', ... })
const humanExps = db.prepare('SELECT * FROM experiences WHERE
contributor_type = ?').all('human')
expect(humanExps.length).toBeGreaterThan(0)

// Step 10: Milestones fire
const milestones = db.prepare('SELECT * FROM milestones WHERE
```

```
operator_pubkey = ?').all(opKey.publicKey)
expect(milestones.length).toBeGreaterThan(0)
```

Final Gap Fill: Hard Rules from Design Doc

Bundle Size Constraint

```
// bundle-size.test.ts
// Test 1: @serendip/protocol bundle stays under 50KB
import { execSync } from 'child_process'
const result = execSync('cd packages/protocol && bun build src/index.ts --
outdir dist --minify').toString()
const { size } = statSync('packages/protocol/dist/index.js')
expect(size).toBeLessThan(50 * 1024)  // 50KB hard cap
```

---

§10 Fairness Charter Immutability

```
// charter.test.ts
// Test 1: §10 exists in protocol spec
const spec = readFileSync('docs/spec/serendip-protocol-v1.md', 'utf8')
expect(spec).toContain('§10')
expect(spec).toContain('cannot be modified')
expect(spec).toContain('SIP')

// Test 2: Anti-gaming rules enforced at relay level (not optional config)
// Same-operator verification = 0 points — hard-coded, not configurable
const scoringCode =
readFileSync('packages/supernode/src/agentxp/scoring.ts', 'utf8')
expect(scoringCode).toContain('same_operator')
expect(scoringCode).not.toContain('configurable')  // must not be a config
option

// Test 3: §10 referenced in CONTRIBUTING.md — contributors know it's
immutable
const contributing = readFileSync('CONTRIBUTING.md', 'utf8')
expect(contributing).toContain('§10')
expect(contributing).toContain('immutable')
```

Serendipity Search is Batch (Not Real-Time)

```
// serendipity-batch.test.ts
// Test 1: Serendipity results come from scheduled batch job, not on-demand
const searchCode = readFileSync('packages/supernode/src/agentxp/experience-
search.ts', 'utf8')
// Serendipity results must be pre-computed and cached, not computed live
expect(searchCode).toContain('serendipity_cache')

// Test 2: Batch job scheduled (e.g. every hour)
const cronConfig = readFileSync('packages/supernode/src/app.ts', 'utf8')
expect(cronConfig).toMatch(/serendipity.*cron|cron.*serendipity/i)

// Test 3: Serendipity search reads from cache, fast response
const start = Date.now()
const results = await search({ query: 'test', channels: { serendipity: true
} })
const duration = Date.now() - start
expect(duration).toBeLessThan(200)  // reads from cache, not computing live
```

Branch Protection Verification

```
// ci-config.test.ts
// Test 1: CI config exists
expect(existsSync('.github/workflows/pr.yml')).toBe(true)
expect(existsSync('.github/workflows/release.yml')).toBe(true)

// Test 2: PR workflow requires all checks
const prWorkflow = readFileSync('.github/workflows/pr.yml', 'utf8')
expect(prWorkflow).toContain('bun test')
expect(prWorkflow).toContain('--frozen-lockfile')
expect(prWorkflow).toContain('typecheck')
expect(prWorkflow).toContain('integration')

// Test 3: Release workflow includes provenance attestation
const releaseWorkflow = readFileSync('.github/workflows/release.yml',
'utf8')
expect(releaseWorkflow).toContain('provenance')

// Test 4: SIP issue template exists
expect(existsSync('.github/ISSUE_TEMPLATE/sip.md')).toBe(true)
const sip = readFileSync('.github/ISSUE_TEMPLATE/sip.md', 'utf8')
expect(sip).toContain('§10')
expect(sip).toContain('backward compat')
```

Pre-coding Checklist Verification

```
// pre-coding-checklist.test.ts
// These verify the checklist items from the design doc appendix were done

// Test 1: @serendip org exists on npm (placeholder published)
// (manual verification — document in CONTRIBUTING.md)
const contributing = readFileSync('CONTRIBUTING.md', 'utf8')
expect(contributing).toContain('@serendip')
expect(contributing).toContain('npm')

// Test 2: SECURITY.md exists with vulnerability disclosure process
expect(existsSync('SECURITY.md')).toBe(true)
const security = readFileSync('SECURITY.md', 'utf8')
expect(security).toContain('vulnerability')
expect(security).toContain('disclosure')

// Test 3: kind-registry repo referenced in CONTRIBUTING.md
expect(contributing).toContain('kind-registry')

// Test 4: PR template exists
expect(existsSync('.github/PULL_REQUEST_TEMPLATE.md')).toBe(true)
const prTemplate = readFileSync('.github/PULL_REQUEST_TEMPLATE.md', 'utf8')
expect(prTemplate).toContain('tests')
expect(prTemplate).toContain('CHANGELOG')
```

All design document hard rules now have corresponding TDD tests. Total spec coverage: complete.